অধ্যায় ২

Variables, Data Types এবং Memory

অধ্যায়ের লক্ষ্য

এই অধ্যায় শেষে তুমি `let`, `const`, `var`, JavaScript-এর Primitive ও Reference Values, `typeof`, Type Conversion, Truthy/Falsy, Scope, `null` / `undefined`, Memory Reference এবং সাধারণ Mutation Bugs বুঝতে পারবে। শেষে সম্পূর্ণ **Student Result Calculator** তৈরি করবে।

২.১ Variable আসলে কী?

Variable হলো একটি নামযুক্ত Binding, যার মাধ্যমে Program কোনো Value ব্যবহার করে। JavaScript-এর Variable-কে শুধু "Memory Box" ভাবলে কিছু ক্ষেত্রে ভুল ধারণা তৈরি হয়; Object-এর ক্ষেত্রে Variable Object-এর Reference ধরে রাখতে পারে।

```javascript
let seats = 12;
seats = 10; // reassignment allowed
const destination = "Dhaka";
// destination = "Sylhet"; // TypeError
```

Naming: `camelCase` ব্যবহার করো; `totalPrice`, `studentName` অর্থবহ। নামের শুরুতে সংখ্যা রাখা যায় না, তবে `_` ও `$` অনুমোদিত। Reserved Keyword (`const`, `return`) নাম হতে পারে না।

২.২ `let`, `const`, `var` — পার্থক্য

বৈশিষ্ট্য	`let`	`const`	`var`
Scope	Block	Block	Function / Global
Reassignment	হ্যাঁ	না	হ্যাঁ
Same scope-এ redeclare	না	না	হ্যাঁ
Initialization বাধ্যতামূলক	না	হ্যাঁ	না
Hoisting behavior	TDZ	TDZ	`undefined` initialization

```javascript
let count = 1;
if (true) {
  let count = 2; // different block-scoped binding
  console.log(count); // 2
}
console.log(count); // 1
```

Recommended Practice: Default হিসেবে `const`; Value পুনরায় Assign করতে হলে `let`; নতুন Code-এ `var` এড়িয়ে চলো। `const` মানে Object Deeply Immutable নয়।

```javascript
const profile = {name: "Rafi"};
profile.name = "Rafi Ahmed"; // allowed
// profile = {}; // TypeError
```

২.৩ JavaScript Data Types

JavaScript-এর সাতটি Primitive Type: `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`; আর Objects-এর মধ্যে Array, Function, Date ইত্যাদি পড়ে। `null`-এর `typeof` ফল `"object"` একটি ঐতিহাসিক Language Quirk।

```javascript
const city = "Dhaka";               // string
const fare = 65000;                 // number
const largeId = 9007199254740993n; // bigint
const available = true;             // boolean
let coupon;                         // undefined
const missing = null;               // null
const key = Symbol("key");         // symbol
const student = {name: "Mina"};     // object
console.log(typeof missing);        // "object" (historical quirk)
console.log(Array.isArray([1, 2])); // true
```

`number` এবং Precision

JavaScript-এর সাধারণ `number` IEEE-754 Double-Precision Floating-Point ব্যবহার করে। তাই:

```javascript
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(Number.isNaN(Number("abc"))); // true
console.log(Number.isFinite(Infinity)); // false
```

Money Calculation-এ Minor Unit (যেমন পয়সা) বা উপযুক্ত Decimal Library বিবেচনা করা ভালো। `BigInt` বড় Integer-এর জন্য, কিন্তু `number`-এর সঙ্গে সরাসরি Arithmetic করা যায় না।

২.৪ `undefined`, `null`, `NaN`

- `undefined`: Value Assign করা হয়নি বা Property নেই।

- `null`: ইচ্ছাকৃতভাবে "কোনো Value নেই" বোঝাতে ব্যবহৃত।
- `NaN`: Invalid Numeric Result; `typeof NaN` হলো `"number"`।

```javascript
let email;
const middleName = null;
const badNumber = Number("hello");
console.log(email === undefined); // true
console.log(middleName === null); // true
console.log(Number.isNaN(badNumber)); // true
```

২.৫ Type Conversion বনাম Coercion

Explicit Conversion-এ Programmer ইচ্ছাকৃতভাবে Type বদলায়; **Implicit Coercion**-এ Operator বা Context পরিবর্তন ঘটায়।

```javascript
Number("42");       // 42
String(42);         // "42"
Boolean(0);         // false
"5" + 2;            // "52" (concatenation)
"5" - 2;            // 3 (numeric coercion)
Number("");         // 0 — input validation-এ বিপজ্জনক!
Number("abc");      // NaN
```

Form Input থেকে সব Value সাধারণত String হিসেবে আসে। `Number("")` হলো `0`, তাই Empty Input আগে আলাদাভাবে Reject করতে হবে।

২.৬ Truthy এবং Falsy

`false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN` হলো Falsy। অন্য সব Object (খালি Array বা Object-সহ) Truthy।

```javascript
console.log(Boolean([]));  // true
console.log(Boolean({}));  // true
console.log(Boolean("0")); // true
console.log(Boolean(0));   // false
```

Common Mistake: `if (marks)` ব্যবহার করলে `0` নম্বরকে Missing ধরে নিতে পারো। বরং `marks === ""` দিয়ে Empty Field পরীক্ষা করো।

২.৭ Scope, Hoisting এবং Temporal Dead Zone

`let` ও `const` Block-Scoped। Declaration-এর আগে একই Scope-এ Access করলে `ReferenceError` হয়—এই অবস্থাকে Temporal Dead Zone (TDZ) বলা হয়।

```javascript
function demo() {
  // console.log(score); // ReferenceError
  const score = 95;
  if (score > 80) {
    const message = "Excellent";
    console.log(message);
  }
  // console.log(message); // ReferenceError
}
```

`var` Function-Scoped এবং Declaration-এর আগে Access করলে অনেক ক্ষেত্রে `undefined` পাওয়া যায়; তাই পুরোনো Code-এ সূক্ষ্ম Bug তৈরি হতে পারে।

২.৮ Primitive Value এবং Object Reference

Primitive Assignment সাধারণত Value Copy-এর মতো আচরণ করে। Object Assignment-এ দুই Binding একই Object-কে Refer করতে পারে।

```javascript
let a = 5;
let b = a;
b = 9;
console.log(a); // 5

const first = {name: "Nila"};
const second = first;
second.name = "Nila Islam";
console.log(first.name); // "Nila Islam"
```

Shallow Copy তৈরি করতে Spread Syntax ব্যবহার করা যায়, কিন্তু Nested Object Shared থাকতে পারে:

```javascript
const original = {name: "A", address: {city: "Dhaka"}};
const copy = {...original};
copy.address.city = "Khulna";
console.log(original.address.city); // "Khulna"
const deepCopy = structuredClone(original); // supported modern runtimes
```

Memory Note: JavaScript Runtime সাধারণত Garbage Collection দিয়ে Unreachable Objects-এর Memory পুনরুদ্ধার করে। `const` Variable-কে Permanent Memory Allocation ভাবা ভুল।

২.৯ `===`, `??` এবং Optional Chaining-এর পরিচয়

```javascript
console.log(0 === "0"); // false
console.log(0 == "0");  // true (coercion)
const preferredSeats = 0;
console.log(preferredSeats ?? 2); // 0
console.log(preferredSeats || 2); // 2
const user = {profile: null};
console.log(user.profile?.email ?? "Not provided");
```

সাধারণত `===` ও `!==` ব্যবহার করো। `??` শুধু `null` বা `undefined` হলে Fallback নেয়; `||` সব Falsy Value-তে Fallback নেয়।

২.১০ Practical Project — Student Result Calculator

Requirements: Student Name, বাংলা, ইংরেজি ও গণিতের নম্বর নিতে হবে। প্রতিটি নম্বর ০–১০০-এর মধ্যে Integer হতে হবে। মোট, গড়, পাস/ফেল ও Grade দেখাতে হবে। কোনো একটি বিষয়ে ৩৩-এর নিচে হলে Overall Fail। Grade Policy এখানে শেখার জন্য উদাহরণ; এটি কোনো নির্দিষ্ট Education Board-এর Official Grading Standard নয়।

Project Files:

```
source-code/chapter-02/
├── index.html
├── style.css
├── calculator.js
├── app.js
└── calculator.test.js
```

Logic Breakdown: `calculateResult(name, marks)` একটি Pure Function; এটি DOM-এর ওপর নির্ভর করে না। `app.js` Form-এর Data নিয়ে Function Call করে। `textContent` দিয়ে Result দেখানো হয় যাতে User Input HTML হিসেবে Execute না হয়।

calculator.js — Core Logic

```javascript
"use strict";
function calculateResult(name, marks) {
  if (typeof name !== "string" || name.trim().length === 0 || name.trim().length > 80) {
    throw new TypeError("শিক্ষার্থীর নাম ১-৮০ অক্ষরের হতে হবে।");
  }
  if (!Array.isArray(marks) || marks.length !== 3 || marks.some(
    (mark) => typeof mark !== "number" || !Number.isInteger(mark) || mark < 0 || mark > 100
  )) {
    throw new RangeError("প্রতিটি বিষয়ে ০-১০০-এর মধ্যে পূর্ণসংখ্যা দাও।");
  }
  const total = marks.reduce((sum, mark) => sum + mark, 0);
  const average = total / marks.length;
  const passed = marks.every((mark) => mark >= 33);
  let grade;
```

```
  if (!passed) grade = "F";
  else if (average >= 80) grade = "A+";
  else if (average >= 70) grade = "A";
  else if (average >= 60) grade = "A-";
  else if (average >= 50) grade = "B";
  else if (average >= 40) grade = "C";
  else if (average >= 33) grade = "D";
  else grade = "F";
  return {name: name.trim(), marks: [...marks], total, average, passed, grade};
}
```

Complete `index.html`, `style.css`, `app.js` ও Test Code একই Package-এর `source-code/chapter-02/` Folder-এ রয়েছে। `index.html` Browser-এ খুলে Application চালাও।

Input → Processing → Output

```
Form Input → Empty Check → Number Conversion → Validation
     → Total / Average / Pass → Grade → Safe DOM Output
```

Example: `Sara`, `[80, 90, 70]` → Total `240`, Average `80.00`, Passed `true`, Grade `A+`। `Sara`, `[100, 100, 20]` → Total `220`, কিন্তু Math-এ Fail হওয়ায় Grade `F`।

২.১১ Debugging Checklist

1. Input Empty থাকলে `Number("")` দিয়ে `0` হয়ে যাচ্ছে কি?
2. Grade-এর আগে সব বিষয়ে Pass Condition যাচাই হচ্ছে কি?
3. `NaN`, Negative বা `101` Reject হচ্ছে কি?
4. User Name Output-এ `textContent` ব্যবহার করা হচ্ছে কি?
5. `const` Object-এর Nested Property পরিবর্তন হলে কেন হচ্ছে বুঝতে পারছ কি?

২.১২ Practice Exercises

Exercise 1: `const` ও `let`-এর মধ্যে Reassignment-এর পার্থক্য Code দিয়ে দেখাও।

Exercise 2: `"25"`, `25`, `null`, `undefined`, `[]` এবং `NaN`-এর `typeof` বের করো।

Exercise 3: কেন `Number("")` Form Validation-এ Problem তৈরি করে? একটি Guard লেখো।

Exercise 4: `const settings = {theme: {mode: "light"}}`-এর Shallow Copy বানিয়ে Nested Mode বদলাও। Original কেন বদলায় ব্যাখ্যা করো।

Exercise 5: Calculator-এ একটি Subject-এ ৩২ ও অন্য দুটিতে ৯০ থাকলে Final Grade কী হবে? কেন?

Exercise 6 (Challenge): Calculator-এ ৪টি Subject Support করতে Function-টিকে Generalize করো; তবে Empty List Reject করবে।

২.১৩ Exercise Solutions

Solution 1:

```javascript
let points = 10;
points = 20; // works
const maxPoints = 100;
// maxPoints = 200; // TypeError
```

Solution 2: যথাক্রমে `"string"`, `"number"`, `"object"`, `"undefined"`, `"object"`, `"number"`। Array-এর জন্য `Array.isArray()` দরকার।

Solution 3:

```javascript
function parseMark(raw) {
  if (raw.trim() === "") throw new Error("Required");
  const value = Number(raw);
  if (!Number.isInteger(value) || value < 0 || value > 100) throw new Error("Invalid mark");
  return value;
}
```

Solution 4: Spread Copy শুধু Top Level Copy করে; Nested `theme` Object Shared থাকে।

```javascript
const settings = {theme: {mode: "light"}};
const shallow = {...settings};
shallow.theme.mode = "dark";
console.log(settings.theme.mode); // "dark"
```

Solution 5: `F`; কারণ যেকোনো Subject-এ ৩৩-এর কম হলে Fail Policy প্রয়োগ করা হয়েছে।

Solution 6 (One Approach):

```javascript
function summarizeMarks(marks) {
  if (!Array.isArray(marks) || marks.length === 0 ||
      marks.some((n) => !Number.isInteger(n) || n < 0 || n > 100)) {
    throw new RangeError("Invalid marks");
  }
  const total = marks.reduce((sum, n) => sum + n, 0);
  return {total, average: total / marks.length, passed: marks.every((n) => n >= 33)};
}
```

২.১৪ Quick Quiz — ১০টি প্রশ্ন

1. `const`-এর Object Property পরিবর্তন করা যায়? **(A) কখনো না (B) হ্যাঁ (C) শুধু Strict Mode-এ**
2. `typeof null` কী? **(A) null (B) undefined (C) object**
3. `Number("")` কত? **(A) 0 (B) NaN (C) undefined**

4. `Boolean([])` কী? **(A) false (B) true (C) error**
5. `"5" + 2` কী? **(A) 7 (B) "52" (C) NaN**
6. `0 ?? 10` কী? **(A) 0 (B) 10 (C) null**
7. `0 || 10` কী? **(A) 0 (B) 10 (C) false**
8. `Number.isNaN(NaN)` কী? **(A) true (B) false (C) error**
9. `let`-এর Scope সাধারণত কী? **(A) Function only (B) Block (C) Always global**
10. `const x = [1]`-এর পর `x.push(2)` কি সম্ভব? **(A) না (B) হ্যাঁ (C) শুধু Node.js-এ**

Quiz Answer Key ও Explanation

প্রশ্ন	উত্তর	ব্যাখ্যা
1	B	`const` Binding স্থির, Object Contents নয়।
2	C	Historical Quirk।
3	A	Empty String Numeric Conversion-এ Zero।
4	B	Empty Array-ও Truthy Object।
5	B	`+` String Concatenation করে।
6	A	`??` Zero-কে Missing ধরে না।
7	B	`||` Zero-কে Falsy ধরে।
8	A	`Number.isNaN` সঠিক NaN Detection।
9	B	`let` Block Scoped।
10	B	Array Contents Mutable।

২.১৫ অধ্যায়ের সারসংক্ষেপ

Variable Binding, Primitive বনাম Object Reference, Scope, Type Conversion ও Input Validation বুঝলে JavaScript-এর অনেক Common Bug আগেই এড়ানো যায়। এই অধ্যায়ের Calculator Project পরের অধ্যায়ের Operators ও Decision Making-এর ভিত্তি তৈরি করে।

পরবর্তী অধ্যায়: Operators, Expressions এবং Decision Making।